

GitHub Copilot 導入プロジェクト

プロジェクト・ロードマップ / GitHub Copilot と Copilot の比較

作成日：2026年10月6日
 対象：社内システム開発部
 GitHub Copilot
 立ち上げプロジェクトチーム

本資料の目次

- ・ 1. 現状整理と、経験の浅いリーダーが取るべきスタンス
- ・ 2. プロジェクト・ロードマップ（全体像 / ガントチャート / フェーズ詳細）
- ・ 3. 最初の30日アクションプラン
- ・ 4. GitHub Copilot と Copilot（Microsoft 365 Copilot）の比較表
- ・ 5. Copilot研修の経験を活かせるポイント
- ・ 6. リスクと対策 / 成功指標（KPI）

1. 現状整理と、リーダーとしてのスタンス

項目	内容
これまで	社内AI推進担当として Copilot（Microsoft 365 Copilot 等）の研修を担当
現在	社内システム開発部へ異動し、GitHub Copilot 立ち上げプロジェクトのリーダーに就任
チーム	開発経験5年以上のベテラン社員で構成
課題	自身の開発経験が1年弱であり、プロジェクトの進め方・行動の起こし方に不安がある

結論：「技術で一番になる」必要はありません。リーダーの役割は「技術の答えを出す人」ではなく「チームが答えを出せる場をつくり、導入を前に進める人」です。技術判断はベテランに任せ、あなたは目的設定・合意形成・推進管理・社内展開・効果測定を担う――この役割分担を最初に明言することが、不安を解消する一番の近道です。

経験の浅いリーダーが意識すべき5つの行動原則

原則	具体的な行動
① 役割を言語化する	キックオフで「技術判断はみなさん、推進と調整は私」と宣言する。リーダーの責任範囲（スケジュール・予算・ルール・社内展開）を明確化。
② 聞く・頼る	全メンバーと1on1（各30分）を実施し、現場の課題・懸念・期待をヒアリング。技術面の相談役として「テックリード」を1名指名する。
③ 自分も手を動かす	自らGitHub Copilotを毎日使い、学習ログを残す。「初心者目線の使いやすさ」は、ベテランにはない貴重な視点としてチームに還元できる。
④ 強みで貢献する	研修設計・教材作成・社内啓発・効果測定は、あなたが最も経験のある分野。ここでチームに価値を出せば信頼は自然と得られる。
⑤ 小さく始め、事実で語る	全社展開の前にパイロットで数値（生産性・満足度）を取り、意思決定をデータで行う。意見の対立も「試して測る」で解決できる。

2. プロジェクト・ロードマップ（全体像：約6か月）

フェーズ	1月目	2月目	3月目	4月目	5月目	6月目
Phase 0 準備・体制づくり						
Phase 1 現状把握・計画策定						
Phase 2 環境・ルール整備						
Phase 3 パイロット導入						
Phase 4 評価・改善						
Phase 5 本格展開・定着化						
◆ マイルストーン	◆ キックオフ	◆ 計画承認	◆ パイロット開始	◆ 中間レビュー	◆ 展開判断	◆ 定着・振返り

※ 期間は目安です。ライセンス調達・セキュリティ審査・規程改定の社内リードタイムに応じて調整してください。審査に時間がかかる会社では Phase 2 を前倒しで並行着手するのが効果的です。

フェーズ別の目的と成果物

フェーズ	目的	主な成果物	完了の判断基準
Phase 0 準備・体制づくり（～2週）	チームの役割分担と進め方の合意	プロジェクト憲章、体制図（RACI）、会議体設計	キックオフ完了、テックリード決定
Phase 1 現状把握・計画策定（～6週）	現場課題の把握と導入目的・KPIの設定	ヒアリング結果、導入目的書、KPI定義、ベースライン測定値	経営/上長による計画承認
Phase 2 環境・ルール整備（～8週）	安全に使える環境と社内ルールの整備	利用ガイドライン、セキュリティ審査結果、ライセンス契約、IDE設定手順	情報セキュリティ部門の承認
Phase 3 パイロット導入（～8週）	小規模チームで効果と課題を検証	パイロット計画、操作研修資料、プロンプト事例集、週次ログ	パイロット期間完了・データ取得
Phase 4 評価・改善（～6週）	効果測定とガイドライン・教材の改善	効果測定レポート、改善版ガイドライン、展開計画	本格展開の可否判断
Phase 5 本格展開・定着化（～8週～）	全開発者への展開と継続的な活用促進	全社研修、FAQ、社内コミュニティ、定期レポート	利用率・満足度がKPIを達成

フェーズ別の詳細タスクと「リーダー（あなた）の行動」

フェーズ	主なタスク（チームで実施）	リーダーの行動（あなたが主導）	ベテランに任せること
Phase 0 準備・体制	・プロジェクト目的の確認 ・メンバーの役割分担 ・定例会議の設定（週1回30分）	・キックオフの企画と進行 ・1on1で各メンバーの期待・懸念を聞く ・テックリードを指名し相談ルートを確保	・技術的な前提条件の洗い出し
Phase 1 現状把握・計画	・開発プロセスと課題の棚卸し ・対象言語/IDE/リポジトリの把握 ・現状の生産性ベースライン測定	・導入目的とKPIを文書化し上長と合意 ・ステークホルダー（情シス・セキュリティ・法務）マップ作成 ・予算・スケジュールの取りまとめ	・どの業務・工程に効くかの評価 ・技術的な測定方法の設計
Phase 2 環境・ルール	・プラン選定（Business / Enterprise） ・組織ポリシー設定（公開コード一致の候補ブロック、モデル/機能の有効化範囲 等） ・IDE・プロキシ環境の確認	・利用ガイドライン（入力禁止情報、生成コードのレビュー義務、ライセンス確認）の起草 ・セキュリティ/法務の審査窓口 ・ライセンス調達手続き	・ポリシー設定の技術的妥当性確認 ・コードレビュー基準への反映
Phase 3 パイロット	・パイロットチーム（5～10名）で実業務に適用 ・補完 / Chat / エージェント機能の活用試行 ・週次で効果・課題を記録	・研修（操作入門・プロンプトのコツ・注意点）の設計と実施 ← あなたの最大の強み ・週次振り返りのファシリテーション ・好事例の収集と共有	・高度な活用法（テスト生成、リファクタ、レビュー支援）の検証 ・現場目線での品質評価
Phase 4 評価・改善	・KPIの測定と比較 ・アンケート（満足度・体感効果） ・課題とリスクの整理	・効果測定レポートを作成し経営層へ報告 ・ガイドライン・教材の改訂 ・本格展開計画と費用対効果の提示	・品質・セキュリティ面の評価 ・技術的改善提案
Phase 5 展開・定着	・段階的なライセンス配布 ・社内コミュニティ・相談窓口の運営 ・定期的な利用状況モニタリング	・全社研修と社内広報（事例発表会など） ・チャンピオン（推進役）制度の立ち上げ ・四半期ごとの効果レポート	・各チームのチャンピオン役 ・活用ノウハウの継続発信

3. 最初の30日アクションプラン

不安なときほど「最初の一步」を具体化することが大切です。以下は着任直後から実行できる行動計画です。

時期	やること	ねらい
1週目	・ 上長と期待値の確認（ゴール、期限、予算、裁量範囲） ・ GitHub Copilot を自分で使い始める（毎日15分） ・ メンバー全員と1on1の日程調整	自分の責任範囲を明確にし、当事者として製品を理解する
2週目	・ 1on1の実施（質問例：今の開発で一番時間がかかる作業は？AIへの期待と懸念は？） ・ テックリードの指名と相談ルールの合意 ・ キックオフ実施（目的・役割・進め方の共有）	ベテランの知見と本音を引き出し、巻き込む
3週目	・ ヒアリング結果の整理と共有 ・ 導入目的・KPI案の作成 → チームでレビュー ・ 情報セキュリティ・法務部門への事前相談	「チームで決めた計画」にし、社内関係者の早期合意を得る
4週目	・ プロジェクト計画書（ロードマップ・体制・予算・リスク）の完成 ・ 上長・関係部署への計画説明と承認取得 ・ パイロットチーム候補の選定	以降のフェーズを正式に動かせる状態にする

ベテランとの関わり方 — 使える言葉の例

場面	フレーズ例
技術判断を委ねる	「この点は〇〇さんの判断が一番確かなので、方針を決めていただけますか。私は決まった内容を関係部署に通して進めます。」
分からないことを聞く	「私の理解が追いついていないので教えてください。研修資料や説明資料に落とし込むときの参考にしたいです。」
意見が割れたとき	「どちらも理由があるので、パイロットで両方試して数字で比べませんか。」
懸念・反発があるとき	「その懸念はガイドラインに反映したいです。どんなルールがあれば安心して使えますか。」

ポイント：「教えてもらう」ことは弱みではなく、ベテランの専門性を尊重する姿勢として受け止められます。決めたことは必ず実行し、進捗が見える化することで、リーダーとしての信頼は「技術力」ではなく「推進力」で築かれます。

4. GitHub Copilot と Copilot (Microsoft 365 Copilot) の比較表

※ ここでの「Copilot」は、これまで研修で扱ってきた業務向けの Microsoft 365 Copilot (Word / Excel / PowerPoint / Outlook / Teams 等で使うAIアシスタント) を想定しています。料金・機能は頻繁に更新されるため、導入時は必ず公式情報をご確認ください。

比較項目	GitHub Copilot	Copilot (Microsoft 365 Copilot)
提供元	GitHub (Microsoft 傘下)	Microsoft
主な対象者	ソフトウェア開発者・エンジニア	一般のビジネスパーソン (全社員)
主な目的	コーディング・開発作業の効率化と品質向上	文書作成・資料作成・メール・会議など事務作業の効率化
利用する場所	VS Code、Visual Studio、JetBrains 系 IDE、Xcode、Eclipse、GitHub.com、GitHub Mobile、ターミナル (CLI) など	Word、Excel、PowerPoint、Outlook、Teams、Loop、Copilot アプリ (Web/デスクトップ/モバイル) など
主な機能	・入力中のコード補完 (インライン提案) ・Copilot Chat (コードの説明・修正・テスト生成) ・エージェントモード / コーディングエージェント (Issue からの実装・PR 作成) ・プルリクエストの要約・コードレビュー支援 ・CLI でのコマンド支援	・文書の下書き・要約・書き換え ・Excel のデータ分析・数式提案 ・PowerPoint の資料自動生成 ・Teams 会議の要約・議事録・ToDo 抽出 ・Outlook のメール下書き・スレッド要約 ・エージェント (Copilot Studio 等) による業務自動化
参照する情報	開いているファイル・リポジトリのコード、Issue/PR、組織のナレッジ (プランによる)	Microsoft Graph 経由の社内データ (メール、予定、チャット、SharePoint/OneDrive の文書) ※ユーザーのアクセス権の範囲内
入力の形	コードそのもの、コメント、自然言語のチャット指示	自然言語のプロンプト (指示文)
出力の形	ソースコード、テストコード、コマンド、説明文、PR	文章、表、スライド、要約、メール文面
法人向けプラン (参考)	Copilot Business / Copilot Enterprise (ユーザー単位の月額課金。個人向けに Free / Pro / Pro+ もあり)	Microsoft 365 Copilot (Microsoft 365 ライセンスへのアドオン、ユーザー単位課金。無料の Copilot Chat もあり)
管理方法	GitHub の Organization / Enterprise 設定でシート割当・ポリシー (機能・モデルの有効化、公開コード一致候補のブロック等) を管理	Microsoft 365 管理センターでライセンス割当、Entra ID・Purview 等で権限・情報保護を管理
セキュリティ・注意点	・生成コードの脆弱性・ライセンス (著作権) 確認 ・秘密情報 (APIキー等) をコードに含めない ・人によるコードレビューを必須とする	・社内データのアクセス権設定の不備による情報露出 (過剰共有) ・機密ラベル・DLP の整備 ・生成内容の事実確認 (ハルシネーション)
効果の測り方 (例)	コード提案の採用率、PR のリードタイム、レビュー時間、開発者満足度、アクティブ利用率	資料作成・メール対応の時間削減、会議後作業の削減、利用率、満足度
研修の重点	IDE での操作、プロンプト (コメント) の書き方、Chat の活用、生成コードのレビュー観点	プロンプトの基本、アプリ別の使い方、業務シナリオ、情報の取り扱い

一言でいうと : GitHub Copilot は「開発者のためのペアプログラマー」、Microsoft 365 Copilot は「全社員のための業務アシスタント」。どちらも「AIの提案を人が確認して使う」という考え方は共通です。

5. Copilot研修の経験を活かせるポイント

開発経験が浅くても、あなたには「AIツールを組織に定着させた経験」という、ベテラン開発者が持っていない強みがあります。

これまでの経験	GitHub Copilot プロジェクトでの活かし方
研修の企画・実施	パイロット/全社向けの研修カリキュラム設計、ハンズオン運営
プロンプト指導	「良い指示の出し方」の考え方はコードでも共通。コメント駆動の提案や Chat の質問の型を教材化
利用ルール・注意点の周知	利用ガイドライン（入力禁止情報、レビュー義務等）の作成と浸透
利用促進・社内広報	事例発表会、社内ポータル、チャンピオン制度の運営
効果測定・報告	KPI 設計、アンケート、経営層向けレポートの作成
つまずきポイントの把握	初心者がどこで困るかを知っていることは、全社展開時に大きな武器になる

6. リスクと対策

リスク	影響	対策
ベテランが利用に消極的	利用率が伸びない	強制せず、効果が出やすい作業（テスト・定型コード・ドキュメント）から始める。懸念をガイドラインに反映
生成コードの品質・脆弱性	不具合・セキュリティ事故	人によるレビュー必須化、静的解析・シークレットスキャンの併用
著作権・ライセンス問題	法的リスク	公開コードと一致する候補のブロック設定、法務との事前合意
機密情報の取り扱い	情報漏えい	利用ガイドラインで入力禁止情報を明示、契約上のデータ取り扱いを確認
効果が見えない	予算の継続が困難	導入前にベースラインを測定し、定量・定性の両面で報告
リーダーの技術理解不足	判断の遅れ	テックリードとの役割分担、自身の継続的な学習（毎日の利用）

成功指標（KPI）の例

区分	指標	目標値（例）
利用	週あたりアクティブ利用率	70%以上
生産性	PR のリードタイム / 定型作業の所要時間	10～20%短縮
品質	レビュー指摘件数・不具合件数	悪化させない（維持～改善）
満足度	開発者アンケート（5段階）	平均4.0以上
定着	社内事例の共有件数	月3件以上

※ 目標値は一例です。Phase 1 で測定したベースラインをもとに、チームで合意して設定してください。

最後に：プロジェクトを成功させるリーダーは、必ずしも最も技術に詳しい人ではありません。目的を示し、メンバーの力を引き出し、決めたことを前に進める人です。あなたの「AIを組織に根付かせた経験」と「初心者目線」は、このプロジェクトに欠かせない強みです。